OtterSec

Monad JIT

Security Assessment

March 20th, 2026 — Prepared by OtterSec

James Wang james.wang@osec.io

SuperFashi superfashi@osec.io

Table of Contents

Executive Summary **3**

Overview 3

Key Findings 3

Scope **4**

Findings **5**

Vulnerabilities **6**

OS-MJT-ADV-00 | Strict Aliasing in Unaligned Read 9

OS-MJT-ADV-01 | Dangling Rvalue Reference Member 10

OS-MJT-ADV-02 | Fragile Cold Storage Gas Accounting 11

OS-MJT-ADV-03 | Incorrect digits10 Buffer Sizing 12

OS-MJT-ADV-04 | Iterator Overrun on Odd Input 14

OS-MJT-ADV-05 | Missing Error Handling After Lookup 15

OS-MJT-ADV-06 | Inconsistency in Pre-Shanghai PUSH0 Encoding 16

OS-MJT-ADV-07 | Inaccurate Weight Updated due to Race Condition 17

OS-MJT-ADV-08 | Incorrect Concurrent Eviction Threshold 18

OS-MJT-ADV-09 | Signed Pointer Bound Comparisons 19

OS-MJT-ADV-10 | Fragile FFI Result Aliasing 21

General Findings **22**

OS-MJT-SUG-00 | Dangling Pointer Risk After Accessor Release 23

OS-MJT-SUG-01 | Unsafe Memory Pool Design 24

OS-MJT-SUG-02 | Busy-Waiting Async Compiler Worker 25

OS-MJT-SUG-03 | Fragile Frame-Specific Memory Cleanup 26

© 2026 Otter Audits LLC. All Rights Reserved.

TABLE OF CONTENTS

OS-MJT-SUG-04 | Missing Validation Logic 27

OS-MJT-SUG-05 | Code Refactoring 30

Vulnerability Rating Scale 32

Procedure 33

© 2026 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Category Labs engaged OtterSec to assess the `jit` program. This assessment was conducted between January 20th and February 27th, 2026. For more information on our auditing methodology, refer to chapter 07.

Key Findings

We produced 17 findings throughout this audit engagement.

In particular, we identified an issue concerning storing pre-blocks as an rvalue reference, as it does not transfer ownership and may leave InferState holding a dangling reference if the original vector goes out of scope (OS-MJT-ADV-01). Also, The cold-access gas logic relies on the minimum gas amount implicitly matching the warm-access cost, which is fragile and tightly couples unrelated gas components, resulting in the gas accounting becoming incorrect if the minimum store gas changes in future revisions (OS-MJT-ADV-02).

Additionally, digits10 is utilized incorrectly as the buffer size, even though it does not represent the maximum number of decimal digits for size_t, potentially resulting in an out-of-bounds scenario in edge cases (OS-MJT-ADV-03), and when the input length is odd, the parser for hex program may advance the iterator by 2 from the last valid character, moving it past end, which is undefined behavior even without dereferencing (OS-MJT-ADV-04).

Furthermore, the assembler assumes offset zero may always be encoded as PUSH0, but that opcode exists only from the Shanghai fork onward. For pre-Shanghai targets this produces invalid bytecode and incorrect label offset calculations (OS-MJT-ADV-06).

We also suggested improving the current memory pool design (OS-MJT-SUG-01), and highlighted several areas that require stricter validation logic to improve robustness and mitigate potential security risks (OS-MJT-SUG-04). Moreover, we made recommendation for refactoring the codebase for better functionality and addressing inconsistencies (OS-MJT-SUG-05).

© 2026 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a git repository at https://github.com/category-labs/monad. This audit was performed against commit c54fc3a.

A brief description of the programs is as follows:

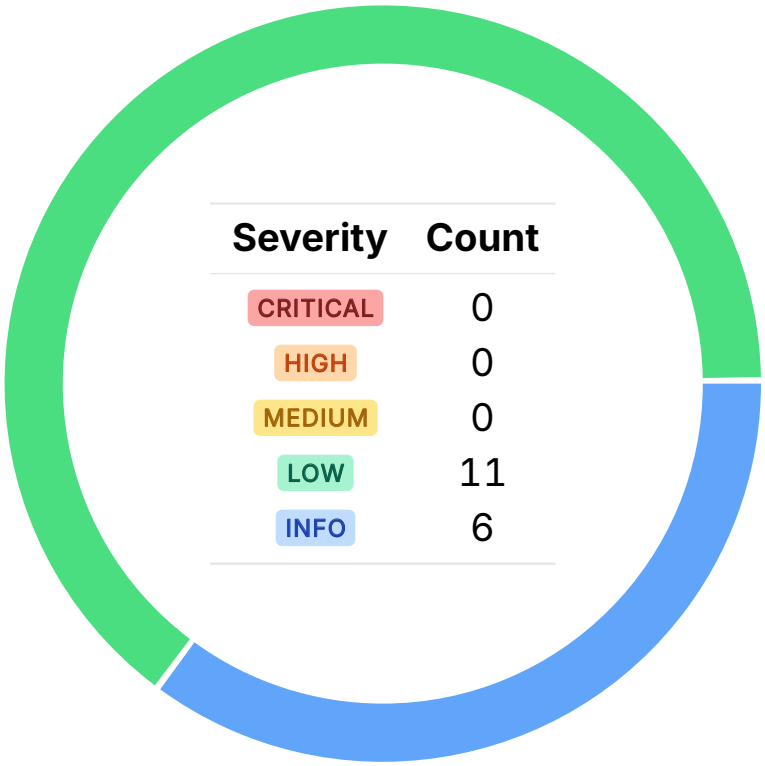
Name	Description
evmc/jit	An EVM implementation that compiles Ethereum bytecode to native machine code utilizing LLVM for faster execution. It exposes an EVMC-compatible VM interface and includes tooling for testing, debugging, and benchmarking compiled contract execution.

© 2026 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 17 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	11
INFO	6

© 2026 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in chapter 06.

ID	Severity	Status	Description
OS-MJT-ADV-00	LOW	RESOLVED	`read_unaligned` avoids unaligned loads, but it still reads a byte buffer through `reinterpret_cast<subword_t*>`, which may violate strict aliasing, resulting in undefined behavior.
OS-MJT-ADV-01	LOW	RESOLVED	Storing `pre_blocks` as an `rvalue` reference does not transfer ownership and may leave `InferState` holding a dangling reference if the original vector goes out of scope.
OS-MJT-ADV-02	LOW	RESOLVED	The cold-access gas logic relies on `min_gas` implicitly matching the warm-access cost, which is fragile and tightly couples unrelated gas components, resulting in the gas accounting becoming incorrect if `minimum_store_gas` changes in future revisions.
OS-MJT-ADV-03	LOW	RESOLVED	`digits10` is utilized incorrectly as the buffer size, even though it does not represent the maximum number of decimal digits for `size_t`, potentially resulting in an out-of-bounds scenario in edge cases.

© 2026 Otter Audits LLC. All Rights Reserved.

OS-MJT-ADV-04	LOW	RESOLVED	When the input length is odd, `parse_hex_program` may advance the iterator by 2 from the last valid character, moving it past end, which is undefined behavior even without dereferencing.
OS-MJT-ADV-05	LOW	RESOLVED	The code logs an error from `Expected<ExecutorAddr>` but still dereferences it utilizing `getValue()`. If the lookup fails, this results in dereferencing an invalid state, resulting in undefined behavior.
OS-MJT-ADV-06	LOW	RESOLVED	The assembler assumes offset zero may always be encoded as `PUSH0`, but that opcode exists only from the Shanghai fork onward. For pre-Shanghai targets this produces invalid bytecode and incorrect label offset calculations.
OS-MJT-ADV-07	LOW	RESOLVED	`insert` may update a node's `cache_weight_` while another thread concurrently evicts the same node and reads that weight for global accounting, creating a race condition that diverges `weight_` from the true total cache weight, resulting in incorrect eviction behavior.
OS-MJT-ADV-08	LOW	RESOLVED	`adjust_by_delta_weight` evicts based on the inserted `delta_weight` instead of the actual amount by which the cache exceeds `max_weight_`, so it may over-evict or still leave the cache overweight. Under concurrency, multiple threads may also evict at once, resulting in unnecessary extra eviction.

© 2026 Otter Audits LLC. All Rights Reserved.

OS-MJT-ADV-09	LOW	RESOLVED	The stack bound helpers utilize signed comparisons for address-like values, which is safe under current user-space memory layouts but implicitly assumes the high bit is never set.
OS-MJT-ADV-10	LOW	RESOLVED	`ffi_runtime` hardcodes `vals[1]` as the return buffer instead of utilizing the computed first-argument index `vals[di]`, so it only works as long as every FFI helper includes `Context*`. It also aliases the return buffer with the first input buffer, creating a fragile assumption that the callee reads its input before writing the result.

© 2026 Otter Audits LLC. All Rights Reserved.

Strict Aliasing in Unaligned Read LOW OS-MJT-ADV-00

Description

`interpreter::detail::read_unaligned` copies bytes into a `std::uint8_t` buffer, but then reads them back via `reinterpret_cast<subword_t*>`, which may still violate C++ strict aliasing rules. This implies the undefined behavior still exists even though the unaligned-access issue is avoided.

>_ *category/vm/interpreter/push.hpp* cpp

```cpp
// We need to do this memcpy dance to avoid triggering UB when
// reading whole words from potentially unaligned addresses in the
// instruction stream. The compilers seem able to optimise this out
// effectively, and the generated code doesn't appear different to
// the UB-triggering version.
[[gnu::always_inline]] inline subword_t
read_unaligned(std::uint8_t const *ptr)
{
    alignas(subword_t) std::uint8_t aligned_mem[sizeof(subword_t)];
    std::memcpy(&aligned_mem[0], ptr, sizeof(subword_t));
    return std::byteswap(
        *reinterpret_cast<subword_t *>(&aligned_mem[0]));
}
```

Remediation

Copy directly into a real `subword_t` local with `std::memcpy` and return that value after byte-swapping. This preserves both alignment safety and aliasing correctness.

Patch

Resolved in 9118ced.

© 2026 Otter Audits LLC. All Rights Reserved.

Dangling Rvalue Reference Member LOW OS-MJT-ADV-01

Description

In `infer_state`, `std::vector<local_stacks::Block> &&pre_blocks` declares a data member that is an `rvalue` reference, not an owned vector. That implies `InferState` does not take ownership of the blocks. it merely aliases some external object that happened to be passed as an `rvalue`. If the constructor receives a temporary object that goes out of scope, the stored reference becomes dangling, resulting in undefined behavior when `InferState` later accesses `pre_blocks`.

>_ *category/vm/compiler/ir/poly_typed/infer_state.hpp* cpp

```cpp
namespace monad::vm::compiler::poly_typed
{
    struct InferState
    {
        std::unordered_map<byte_offset, block_id> const &jumpdests;
        std::vector<local_stacks::Block> &&pre_blocks;
        [...]
    };
}
```

Remediation

The `poly_typed` IR stage is removed since it was never intended to be used in production compile pipelines.

Patch

Resolved in PR#2210.

© 2026 Otter Audits LLC. All Rights Reserved.

Fragile Cold Storage Gas Accounting LOW OS-MJT-ADV-02

Description

The current cold-access charging logic in `storage::sstore` subtracts `traits::cold_storage_cost() + min_gas`, implicitly assuming that `min_gas` is exactly the warm-access component (100) that should already be charged. While this happens to be true under the current EIP-2929 configuration, it creates a dependency between unrelated gas components. This renders the implementation fragile to future refactors or trait changes. If `minimum_store_gas<traits>()` changes in future revisions, the cold-access calculation may become incorrect without obvious detection.

>_ *category/vm/runtime/storage.hpp* cpp

```cpp
template <Traits traits>
void sstore(
    Context *ctx, uint256_t const *key_ptr, uint256_t const *value_ptr,
    std::int64_t remaining_block_base_gas)
{
    [...]
    auto access_status = EVMC_ACCESS_COLD;
    if constexpr (traits::eip_2929_active()) {
        access_status = ctx->host->access_storage(
            ctx->context, &ctx->env.recipient, &key);
        if (access_status == EVMC_ACCESS_COLD) {
            ctx->deduct_gas(traits::cold_storage_cost() + min_gas);
        }
    }
    [...]
}
```

Remediation

Charge the cold surcharge utilizing an explicit constant for the warm access portion, so the formula directly reflects the protocol model. More broadly, the cold/warm storage accounting may be refactored so that access cost and store-transition cost are computed separately.

Patch

The Monad team acknowledges and accepts the risk.

© 2026 Otter Audits LLC. All Rights Reserved.

Incorrect digits10 Buffer Sizing LOW OS-MJT-ADV-03

Description

The buffer size utilized in `new_var` is derived from `std::numeric_limits<size_t>::digits10`, which represents the number of base-10 digits that may be represented without loss of precision, not the maximum number of decimal digits required to print the type's largest value. For a 64-bit `size_t`, `digits10` is typically 19, while the largest value ($2^{64}-1$) requires 20 decimal digits. As a result, allocating `char buffer[digits10]` may theoretically be insufficient to hold the full decimal representation. Although the code increments `ctx.next_subscript` gradually and reaching the absolute maximum `size_t` value is practically unlikely, the implementation still relies on an incorrect bound.

>_ *category/vm/utils/evm-as/compiler.cpp* cpp

```cpp
std::string new_var(annot_context &ctx)
{
    std::stringstream ss{};
    ss << var_letters[ctx.next_letter++];
    ctx.next_letter = ctx.next_letter % var_letters.size();
    constexpr size_t sz = std::numeric_limits<size_t>::digits10;
    char buffer[sz];
    size_t i = 0;
    if (ctx.next_subscript == 0) {
        buffer[i++] = '0';
    }
    else {
        size_t value = ctx.next_subscript;
        while (value > 0) {
            buffer[i++] = static_cast<char>('0' + (value % 10));
            value /= 10;
        }
    }
    for (size_t j = 0; j < i; j++) {
        ss << buffer[i - j - 1];
    }
    if (ctx.next_letter == 0) {
        ctx.next_subscript++;
    }
    return ss.str();
}
```

Remediation

Allocate one additional character (`digits10 + 1`) to guarantee sufficient space for the largest decimal representation. Alternatively, the conversion logic may be simplified by writing the value directly to the `stringstream`, removing the temporary buffer entirely.

© 2026 Otter Audits LLC. All Rights Reserved.

Patch

The Monad team acknowledges and accepts the risk.

© 2026 Otter Audits LLC. All Rights Reserved.

Iterator Overrun on Odd Input LOW OS-MJT-ADV-04

Description

`parse_hex_program` increments `input_it` by 2 each iteration, but when the input length is odd the last valid iteration may start at the final character. Advancing that iterator by 2 moves it past the one-past-the-end position, which is undefined behavior for random-access iterators even if the invalid iterator is never dereferenced. As a result, the intended *"ignore trailing nibble"* behavior is implemented unsafely.

Remediation

Clamp iteration to complete 2-character pairs only, so the iterator never advances beyond end.

Patch

The Monad team acknowledges and accepts the risk.

© 2026 Otter Audits LLC. All Rights Reserved.

Missing Error Handling After Lookup LOW OS-MJT-ADV-05

Description

`LLVMState::set_contract_addr_from_disk` retrieves the JIT entrypoint symbol utilizing `lljit->lookup("contract")`, which returns an `Expected<ExecutorAddr>`. This LLVM type represents either a valid result or an error. The code correctly attempts to extract the error via `takeError()` and prints it if present, but it does not stop execution afterward. If lookup fails, `expected_contract_addr` no longer contains a valid value.

>_ *category/vm/llvm/llvm_state.hpp* cpp

```cpp
void set_contract_addr_from_disk(std::string_view fn)
{
    auto ObjectBuffer = MemoryBuffer::getFile(fn);
    MONAD_VM_ASSERT(ObjectBuffer);
    ExitOnErr(lljit->addObjectFile(std::move(*ObjectBuffer)));
    JITDylib &jd = lljit->getMainJITDylib()

    MONAD_VM_ASSERT(!jd.define(absoluteSymbols(opcode_syms)))

    Expected<ExecutorAddr> expected_contract_addr =
        lljit->lookup("contract")
    if (auto err = expected_contract_addr.takeError()) {
        errs() << "error:" << toString(std::move(err)) << '\n';

    contract_addr = reinterpret_cast<void (*)()>(
        expected_contract_addr->getValue());
};
```

However, the code still proceeds to call `expected_contract_addr->getValue()`, dereferencing an invalid state, resulting in undefined behavior in release builds.

Remediation

The `LLVM` JIT backend is deprecated and removed in favor of the production ready `x86 asmjit` JIT backend.

Patch

Resolved in PR#2211.

© 2026 Otter Audits LLC. All Rights Reserved.

Inconsistency in Pre-Shanghai PUSH0 Encoding LOW OS-MJT-ADV-06

Description

The label-address encoding in `compile` currently treats offset zero as a special case that maps to `PUSH0`. That is only valid on EVM revisions where `PUSH0` exists (Shanghai and later). For pre-Shanghai targets, there is no `PUSH0` opcode, so encoding a label at byte offset zero this way produces invalid bytecode. The same assumption appears in `resolve_labels`, where an offset of zero is assigned width zero, rendering the instruction size `1 + 0 = 1`. Pre-Shanghai, however, pushing zero must still utilize at least `PUSH1 0x00`, which is two bytes total. This results in incorrect label offsets during the fixed-point layout calculation. Additionally, `mcompile` prints `PUSH0` mnemonics even when targeting revisions that do not support the opcode.

Remediation

Update the encoding logic such that it depends on the target `traits`, falling back to `PUSH1 0x00` for pre-Shanghai environments.

Patch

Resolved in PR#2213.

© 2026 Otter Audits LLC. All Rights Reserved.

Inaccurate Weight Updated due to Race Condition LOW OS-MJT-ADV-07

Description

There is potential race condition in `LruWeightCache`. When `insert` updates an existing entry, it reads the old `cache_weight_`, computes `delta_weight`, and then writes the new `cache_weight_`. If another thread evicts that same node concurrently, `evict` may read either the old or new `cache_weight_` while subtracting from the global `weight_`. This implies the per-node weight transition and the global weight accounting are not synchronized as one atomic operation. As a result, `weight_` may drift from the true sum of live entries, resulting in under-eviction or over-eviction.

Remediation

Ensure that the weight change and any possible eviction of that node are mutually consistent.

Patch

The Monad team acknowledges and accepts the risk.

© 2026 Otter Audits LLC. All Rights Reserved.

Incorrect Concurrent Eviction Threshold LOW OS-MJT-ADV-08

Description

`LruWeightCache::adjust_by_delta_weight` checks whether the new total weight exceeds `max_weight_`, but its eviction target is wrong as it evicts until `evicted_weight >= delta_weight` instead of evicting the actual overshoot (`pre_weight + delta_weight - max_weight_`). This implies the cache may evict more entries than necessary, or under concurrent updates, still remain above the configured limit.

>_ *category/vm/utils/lru_weight_cache.hpp* cpp

```cpp
void adjust_by_delta_weight(int64_t delta_weight)
{
    int64_t const pre_weight =
        weight_.fetch_add(delta_weight, std::memory_order_acq_rel);
    if (delta_weight + pre_weight > max_weight_) {
        int64_t evicted_weight = 0;
        while (evicted_weight < delta_weight) {
            ListNode const *target = lru_.evict();
            if (MONAD_VM_UNLIKELY(!target)) {
                break;
            }
            int64_t const n = evict(target);
            weight_.fetch_sub(n, std::memory_order_acq_rel);
            evicted_weight += n;
        }
    }
}
```

Additionally, the function is atomic at the field-update level, but not race-free at the eviction-policy level. Multiple threads may simultaneously observe an overweight cache and each start evicting, so their combined work may remove far more entries than required.

Remediation

Allow only one evictor at a time by utilizing an atomic flag. That single evictor should repeatedly read the live `weight_` and continue evicting while `weight_ > max_weight_`.

Patch

The Monad team acknowledges and accepts the risk.

© 2026 Otter Audits LLC. All Rights Reserved.

Signed Pointer Bound Comparisons LOW OS-MJT-ADV-09

Description

IN `emitter`, `is_stack_overflow` and `is_stack_underflow` currently utilize signed pointer comparisons (`sgt` / `slt`) even though they are really checking raw address bounds. This works in practice as long as all relevant user-space addresses stay below the sign bit, but that is an implicit platform assumption rather than the natural semantics of the check. Thus, utilizing unsigned comparison will better match the intent.

>_ *category/vm/llvm/emitter.hpp* RUST

```
Value *is_stack_overflow(Value *stack_top, int32_t high)
{
    [...]
    if (is_stack_overflow_f == nullptr) {
        [...]
        Value *stack_end = arg[0];
        stack_end->setName("stack_end");
        Value *stack_high = arg[1];
        stack_high->setName("stack_high");
        [...]
    }
    [...]
}

Value *is_stack_underflow(Value *stack_top, int32_t low)
{
    [...]
    if (is_stack_underflow_f == nullptr) {
        [...]
        Value *stack_begin = arg[0];
        stack_begin->setName("stack_begin");
        Value *stack_low = arg[1];
        stack_low->setName("stack_low");
        [...]
    }
    [...]
};
```

Remediation

The `LLVM` JIT backend is deprecated and removed in favor of the production ready `x86 asmjit` JIT backend.

© 2026 Otter Audits LLC. All Rights Reserved.

Patch

Resolved in PR#2211.

© 2026 Otter Audits LLC. All Rights Reserved.

Fragile FFI Result Aliasing LOW OS-MJT-ADV-10

Description

IN `emitter`, `ffi_runtime` selects `vals[1]` as the result buffer when `n != 0`, but the correct first argument slot is actually `vals[di]`, where `di = has_ctx ? 1 : 0`. This is currently safe because all current FFI helpers appear to include `Context*`, so the hardcoded index happens to line up with the first operand.

>_ *category/vm/llvm/emitter.hpp* RUST

```
template <typename... FnArgs>
Function *ffi_runtime(Instruction const &instr, void (*fun)(FnArgs...))
{
    [...]
    if (has_ret) {
        r = n == 0 ? llvm.alloca_(llvm.word_ty, "retval") : vals[1];
        vals.insert(vals.begin() + di, r);
    }
    [...]
}
```

Moreover, the result buffer is aliased to the first input argument buffer, so correctness depends on the callee fully reading that input before writing the result. That is a fragile ABI assumption as a future helper without `Context*`, or one that writes output before consuming input, may silently produce incorrect behavior.

Remediation

The `LLVM` JIT backend is deprecated and removed in favor of the production ready `x86 asmjit` JIT backend.

Patch

Resolved in PR#2211.

© 2026 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MJT-SUG-00	`LruWeightCache` may utilize raw pointers to hash-map entries after releasing the TBB accessor that protects their lifetime.
OS-MJT-SUG-01	The `MemoryPool` exposes raw-pointer allocation and manual management, which risks misuse and weak ownership guarantees. Using RAII (`unique_ptr` / `Ref`), aligned operator new, stronger capacity invariants, and possibly a lock-free free list will improve safety and scalability.
OS-MJT-SUG-02	The async compiler worker relies on timeout-based polling, resulting in unnecessary CPU activity when no jobs exist. Since TBB containers already utilize internal synchronization, replacing the queue with a blocking `tbb::concurrent_bounded_queue` will remove the polling and simplify the concurrency model.
OS-MJT-SUG-03	`Memory::clear` encodes parent/child frame cleanup assumptions that are only correct because of its current narrow call pattern, rendering it fragile if reutilized elsewhere. The extra zeroing of `parent_handle[:parent_capacity]` in the alternate branch also appears unnecessary.
OS-MJT-SUG-04	Several areas require stricter validation logic to improve robustness and mitigate potential security risks.
OS-MJT-SUG-05	Recommendation for refactoring the codebase for better functionality and addressing inconsistencies.

© 2026 Otter Audits LLC. All Rights Reserved.

Dangling Pointer Risk After Accessor Release OS-MJT-SUG-00

Description

`LruWeightCache` sometimes stores a raw `ListNode*` from a `tbb::concurrent_hash_map` entry, releases the accessor, and then continues utilizing that pointer. Since `tbb::concurrent_hash_map` doesn't explicitly guarantee the stability of internal pointers, this is a dangerous pattern that may result in use-after-free if internal pointer management of TBB ever changes.

Remediation

Avoid utilizing map-backed node pointers after releasing the accessor.

© 2026 Otter Audits LLC. All Rights Reserved.

Unsafe Memory Pool Design

OS-MJT-SUG-01

Description

In `MemoryPool`, the current design exposes raw pointers and manual `alloc` / `dealloc`, which makes correct ownership management dependent on callers and increases the risk of misuse such as leaks or returning buffers to the wrong pool. Utilize `std::unique_ptr` (or exposing only the RAII `Ref` wrapper) to enforce automatic return of buffers and safer lifetime management. Also, replace `std::aligned_alloc` with aligned `operator new(std::align_val_t)` to provide a safer allocation model. Additionally, make `alloc_capacity` a template parameter or hide raw deallocation to prevent capacity mismatches, and implement a lock-free free list to reduce contention if the pool is utilized heavily across threads.

Remediation

Implement the above improvements for `MemoryPool`.

© 2026 Otter Audits LLC. All Rights Reserved.

Busy-Waiting Async Compiler Worker

OS-MJT-SUG-02

Description

The async compilation worker currently waits utilizing a condition variable with a 1 ms timeout and repeatedly checks the queue, which effectively introduces periodic polling. This implies the consumer thread wakes up even when no work exists, resulting in unnecessary CPU wakeups and scheduling overhead.

Additionally, the comment suggests avoiding locks on the producer path, but the utilization of `tbb::concurrent_queue` and `tbb::concurrent_hash_map` already introduces internal synchronization, so the implementation in `async_compile` is not truly lock-free.

Remediation

Remove the condition variable and associated mutex locks from the class, and replace `tbb::concurrent_queue` with `tbb::concurrent_bounded_queue`, which supports blocking on the consumer side and eliminates busy waiting.

© 2026 Otter Audits LLC. All Rights Reserved.

Fragile Frame-Specific Memory Cleanup OS-MJT-SUG-03

Description

`Memory::clear` works only because it is utilized in a very specific lifecycle, effectively assuming that it is always called as the top-level `Memory`. This holds true currently because the function is only invoked in `Context::return_to`, where the runtime already knows whether it is unwinding the top-level frame or restoring a parent frame. However, this renders the function fragile, as re-utilizing it in other contexts may result in the `parent_handle == data_handle` check to no longer reflect valid ownership semantics.

>_ *category/vm/runtime/types.hpp* RUST

```rust
[[gnu::always_inline]]
void clear()
{
    if (MONAD_VM_LIKELY(parent_handle == data_handle)) {
        non_temporal_bzero(data_handle, size);
    }
    else {
        non_temporal_bzero(parent_handle, parent_capacity);
        std::free(data_handle);
    }
}
```

Furthermore, the `else` branch sets `parent_handle[:parent_capacity]` to zero even though that parent-backed region is already logically discarded from the current frame's perspective, rendering this operation unnecessary. Therefore it is not required to store `parent_capacity` in `Memory`, and this logic maybe moved into `return_to` directly.

Remediation

Move the parent/child ownership-specific cleanup logic out of `Memory::clear` and into `Context::return_to`, where frame unwinding semantics are already explicit.

© 2026 Otter Audits LLC. All Rights Reserved.

Missing Validation Logic

OS-MJT-SUG-04

Description

1. `MemoryPool` only validates alignment of `alloc_capacity` but does not ensure it is non-zero, allowing a pool to be created with an invalid buffer size. A zero capacity may result in undefined behavior during aligned allocation and break assumptions that the buffer may store the free-list node. Ensure that `alloc_capacity` is non-zero.
2. `build_untyped` assumes that `tb.output` contains at least two elements when handling a `JumpI` terminator, but this assumption is not explicitly validated. Although the compiler pipeline guarantees this invariant during normal execution, constructing `output_tail` with `tb.output.data() + 2` and `tb.output.size() - 2` without checking the size may still result in out-of-bounds memory access if `tb.output.size() < 2`. Explicitly enforce the bounds check to prevent unsafe access.

>_ *category/vm/compiler/ir/untyped.cpp* RUST

```rust
std::variant<std::vector<Block>, std::vector<local_stacks::Block>>
build_untyped(
    std::unordered_map<byte_offset, block_id> const &jumpdests,
    std::vector<poly_typed::Block> &&typed_blocks)
{
    [...]
    else {
        poly_typed::JumpI const &jumpi =
            std::get<poly_typed::JumpI>(tb.terminator);
        std::span<poly_typed::Value> const output_tail(
            tb.output.data() + 2, tb.output.size() - 2);
        [...]
    }
    [...]
}
```

3. Both `local_stacks::eval_instruction_fallback` and `infer::infer_instruction_default` assume that any instruction that increases the stack does so by exactly one element. When `tok.increases_stack()` or `ins.increases_stack()` is true, the code pushes exactly one placeholder (`ValueIs::COMPUTED`) or one word kind. However, the code never verifies that the instruction's actual `stack_increase()` value is indeed 1. Add an assertion such as `tok.stack_increase() == 1` (and similarly for `ins.stack_increase()`) to render this assumption explicit and detect mismatches early. Alternatively, utilize `stack_increase()` directly to push the correct number of elements.

© 2026 Otter Audits LLC. All Rights Reserved.

```rust
>_ category/vm/compiler/ir/local_stacks.cpp
void
eval_instruction_fallback(Instruction const &tok, std::deque<Value> &stack)
{
    for (std::size_t i = 0; i < tok.stack_args(); ++i) {
        stack.pop_front();
    }
    if (tok.increases_stack()) {
        stack.emplace_front(ValueIs::COMPUTED, 0);
    }
}
```

4. In `evm-as`. `PlainI` is intended for fixed-width opcodes, but if a `PUSHn` opcode is mistakenly stored in it, the assembler will emit only the opcode byte and omit the required immediate bytes. As a result, the byte stream becomes malformed even though the assembler itself does not necessarily fail. The emitted output would no longer correspond to the intended instruction sequence. Thus, a `EvmBuilder` misuse may invalidate the rest of the assembled program. Add a guard that rejects `PUSHn` opcodes.
5. `GeoMean::update` calls `std::log2(new_value)` without validating the input, even though geometric means require strictly positive values. Add a validation to sanity check the input.

```rust
>_ category/vm/utils/parser.cpp
template <typename T>
struct GeoMean
{
    std::atomic<T> running_avg_{0};
    std::atomic<T> count_{0};
    void update(T new_value) noexcept
    {
        [...]
        auto new_avg =
            (running_avg * count + std::log2(new_value)) / (count + 1);
        running_avg_.store(new_avg, std::memory_order_release);
        count_.fetch_add(1, std::memory_order_release);
    }
    [...]
};
```

6. In `resolver::resolve_labels`, the `it != label_offsets.end()` is not a meaningful assertion, because `unordered_map::insert` returns a valid iterator whether insertion succeeded or the key already existed. If the goal is to detect duplicate labels, the code should assert on the returned inserted boolean instead.

© 2026 Otter Audits LLC. All Rights Reserved.

>_ *category/vm/utils/evm-as/resolver.hpp* RUST

```rust
template <Traits traits>
std::unordered_map<std::string, size_t>
resolve_labels(EvmBuilder<traits> const &eb)
{
    [...]
    for (Instruction::T const &ins : eb) {
        if (Instruction::is_comment(ins)) {
            continue;
        }
        offset =
            offset +
            std::visit(
                Cases{
                    [&](PushLabelI const &) -> size_t {
                        return 1; // optimistically assume a 1 byte encoding.
                    },
                    [&label_offsets,
                     offset](JumpdestI const &jumpdest) -> size_t {
                        auto const [it, _] =
                            label_offsets.insert({jumpdest.label, offset});
                        MONAD_VM_ASSERT(it != label_offsets.end());
                        return 1; // 1 byte encoding.
                    },
                    [...]
    }
    [...]
}
```

Remediation

Include the validations mentioned above.

© 2026 Otter Audits LLC. All Rights Reserved.

Code Refactoring

OS-MJT-SUG-05

Description

1. `parser::parse_opcodes_helper` relies on a hardcoded bound (33) when searching the `push_ops_with_arg` array, implicitly assuming the array size will never change. It will be more appropriate to utilize iterators or `.size()` to ensure the search respects the actual container bounds and prevents maintenance errors if the array definition is modified.

>_ *category/vm/utils/parser.cpp* RUST

```
std::vector<uint8_t>
parse_opcodes_helper(parser_config const &config, std::string const &str)
{
    [...]
        else if (is_push_with_arg(op)) {
            auto r = parse_constant_or_label(input);
            input = r.first;
            std::visit(
                Cases{
                    [&](runtime::uint256_t const &imm) -> void {
                        auto const *const pushops =
                            push_ops_with_arg.data();
                        auto const d = std::distance(
                            pushops,
                            std::find(pushops, pushops + 33, op));
                       [...]
                    },
                    [&](std::string const &label) -> void {
                        eb.push(label);
                    }},
                r.second);
        }
    [...]
}
```

2. The interpreter relies on the native memory layout when reading or writing multi-byte values, implicitly assuming a little-endian architecture. Consequently, execution on big-endian systems will reverse byte ordering and result in incorrect value interpretation. The implementation should avoid dependence on native integer layout to ensure compatibility across both architectures.
3. The current design lets the LRU list depend on memory owned internally by the concurrent hash map, which makes node lifetime fragile under erase or concurrent mutation. A safer design may be to invert the dependency by having the hash map store a pointer to the corresponding entry in the doubly linked list.

© 2026 Otter Audits LLC. All Rights Reserved.

4. `log_utils` stores the running average in the template type `T`, so if `T` is an integer, each update performs truncated arithmetic rather than preserving fractional precision. This accumulates rounding error over time. Refactor the logic to avoid repeated truncation inside the update formula.
5. Currently in `emitter::gas_decrement_unbounded_work`, `accumulated_static_work_` is reset unconditionally, even though the rest of the function only performs meaningful work when `gas` is nonzero. Thus `accumulated_static_work_ = 0` should ideally be moved inside the `if` condition to ensure the reset occurs only when a real deduction is emitted. However, if `gas` is in practice never zero, it will be more appropriate to remove the `if` condition entirely.

>_ *category/vm/compiler/ir/x86/emitter.cpp* RUST

```
void Emitter::gas_decrement_unbounded_work(int64_t gas)
{
    accumulated_static_work_ = 0;
    if (gas) {
        gas_decrement_no_check(gas);
        as_.jl(error_label_);
    }
}
```

6. `return_data_size` is declared as `std::size_t`, but `emitter::returndatasize` reads it utilizing a helper that only consumes 32 bits, so the access pattern does not clearly match the field type. This is safe currently because the size of `returndata` is effectively bounded by `uint32` and the target is assumed to be little-endian. However, this assumption is implicit, so utilizing a fixed-width field, a corresponding read helper, or at least a clarifying comment explaining why only the lower 32 bits are intentionally read would improve robustness and clarity.

>_ *category/vm/compiler/ir/x86/emitter.cpp* RUST

```
void Emitter::returndatasize()
{
    static_assert(
        sizeof(runtime::Environment::return_data_size) == sizeof(uint64_t));
    read_context_uint32_to_word(
        runtime::context_offset_env_return_data_size);
}
```

Remediation

Incorporate the above refactors.

© 2026 Otter Audits LLC. All Rights Reserved.

06 — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

07 — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.